



General Instrument CP1600 explained

The **CP1600** is a [16-bit microprocessor](#) created in a partnership between [General Instrument](#) and [Honeywell](#), introduced in February 1975. ^[1] ^[2] It is one of the first single-chip 16-bit processors. The overall design bears a strong resemblance to the [PDP-11](#).

[Honeywell](#) used the CP1600 in a number of [process control computers](#) and related systems, but its most widespread use was the **CP1610** version in the [Intellivision video game console](#). The 1610 was essentially a slower version of the 1600, running at a maximum speed around 2 MHz instead of the 3 to 5 MHz of the 1600.

The system saw little other use due to General Instrument's marketing philosophy of seeking out customers only with very large orders and ignoring smaller customers. They also did not pursue a [second source](#) arrangement, which in the early days of microprocessor designs was a requirement for most potential customers.

Description

Physical implementation

The CP1600 was implemented in [enhancement mode nMOS](#) and required +12, +5, and −3 V power supplies; I/O connections except for the clocks were [TTL](#) (5 V) compatible. Each microstate or processor cycle uses four internal time slots generated by two non-overlapping clocks. A 3.3 MHz two-phase clock produces a 600 nanosecond microcycle. A 5 MHz two-phase clock produces a 400 nanosecond microcycle. Due to the voltage requirements of the clock signals, these had to be generated with external circuitry, as was common in this era of microprocessor design.

In order to fit a 16-bit processor into a 40-pin [dual in-line package](#) (DIP) chip design, the CP1600 multiplexed its data and address pins. This allowed a set of 16 pins to be used for both address selection and reading and writing data, but to do so required two bus cycles. It also complicated the overall machine layout as buffers were required on the memory bus to latch the address while the processor switched the pins to data mode. The interface to the system was likewise complex, requiring three pins,, and, which had to be decoded to understand what state the memory bus was in.

A relatively uncommon feature of the CP1600 was its "external branch" concept. The instruction opcode left the lower four bits of the code free, leaving it to the programmer to place a number in those four bits. When the opcode was called, any of those four bits set to high (1) would be expressed as high voltages on the external pins through . This was normally used to select one of up to 16 external devices. The selected device would then respond by setting the to high if it wanted to communicate. If was high, the branch address provided with the would then be taken, if was low, the branch would be skipped. This could be used, for instance, to test whether an external device had input data that needed to be processed; the processor could express the value "2" on the to sample device 2, call the, and that device would then respond by setting to true if there was data, causing the processor to jump into the [device driver](#) code to read the data from that device.

This contrasts with the typical solution for handling external devices; most systems have the devices [raise an interrupt](#) which causes the processor to call special code, the [interrupt handler](#), which then reads additional data to determine which device called the interrupt. This additional data may be presented using dedicated pins on the CPU, but is more commonly presented as a value on the [data bus](#). Based on that value, the interrupt handler code then decides which device driver to call to process the data. The CP1600 can implement this in fewer instructions; the interrupt handler is simply a series of instructions pointing at the associated drivers which it runs through one at a time until the device in question sets the and automatically cause the code to branch.

In total, implementing a system using the CP1600 often required additional support chips and logic. This included a system to multiplex sixteen signals into a single pin if the external branching was being used, and a three-bit-to-eight-line converter to avoid having to decode the bus status signals in external parts.

Instruction set and registers

+ CP1600 registers

$\overset{1}{5} \overset{1}{4} \overset{1}{3} \overset{1}{2} \overset{1}{1} \overset{0}{9} \overset{0}{8} \overset{0}{7} \overset{0}{6} \overset{0}{5} \overset{0}{4} \overset{0}{3} \overset{0}{2} \overset{0}{1} \overset{0}{0}$ (*bit position*)

Main registers

Search

R0	Register 0 / Accum
R1	Register 1 / Indirect
R2	Register 2 / Indirect
R3	Register 3 / Indirect
R4	Register 4 / Autoincrement
R5	Register 5 / Autoincrement
R6 / SP	Register 6 / Stack Pointer
R7 / PC	Register 7 / Program Counter

Status flags

S Z OV C Status Flags

The smallest unit of addressable and writable memory is the 16-bit word; byte addressing is not supported. The CP1600 can address 64K words, or in modern terms, 128 kbytes. Of the 16-bits available in an instruction [opcode](#), the CP1600 used only 10. The remaining 6 bits were marked "Future Use."^[3] The 10-bit instructions meant that code stored in a typical byte-oriented ROM would waste six bits per instruction. In the era of expensive memory, this was a significant issue. To address this, General Instrument also produced special 10-bit ROMs that efficiently stored the instructions. As data and addresses would still require 16-bit values, the 1600 included a special instruction that pieced together a 16-bit argument from two 10-bit ROM reads.

The unused 6 bits were intended to be used with co-processors, asserting the line which stalled the CPU until released. Early documentation shows two planned chips in the series, the 1616 which added the "Extended Instruction Set", and the 1618 "Priority Expander".

The system included 87 basic instructions. Instructions might be one to three words long depending on the addressing format and whether 10-bit or 16-bit program memory was being used. The CP1600 did not support memory-memory indirect addressing (offsets), and indexing was implemented using a dedicated adder that performed single-cycle changes to addresses in registers. The [arithmetic logic unit](#) (ALU) was 8 bits wide and could add two 16-bit internal registers in 2.4 microseconds, and memory to register adds of 16-bit numbers in 3.2 microseconds.

Like the PDP-11, the CP1600 used eight 16-bit "general purpose" [processor registers](#), although they were not truly general-purpose as in modern designs. Only R0 had no pre-defined purpose and has been described as "the primary accumulator".^[4] R1 through R3 could be used data pointers, generally used for register-based addressing ("implied addressing"). R4 and R5 autoincremented after being accessed, which made them useful for looping over collections of data.

R6 was the [stack pointer](#), R7 the [program counter](#). Since both of these registers were visible to the programmer, they could be used to implement multiple stacks, or support more complex branching, among other things. There were no explicit stack or instructions. When R6 was used in a "read" operation it decremented the address and then returned the data being pointed at, the equivalent of a ; when it was used in a "write" it would write then post-increment (like with the autoincrementing registers R4/R5), simulating a .

Example code

The following [assembler](#) source code is for a subroutine CPYMEM that copies words from one location to another. Note that R4 and R5 autoincrement after being accessed. This makes it easy to loop through an array of words. Curiously, the CP1600 has a 16-bit word width but only a 10-bit instruction width. The following assumes 10-bit ROMs are being used.

Assembled by brain so object code errors possible. -->	10001000	
	2841001	Copy memory words addressed by R4 to location
	2451002	addressed
	0131003 22C	• by R5 for a length of R3 words. Return address in R2.
	0041005 097	• @@loop is a local label ROMW 10 ;Assume 10-bit program

```

memoryCPYMEM PROC@@loop: MVI@ R4,R0 ;Get word to
copy: MOV (R4)+,R0 MVO@ R0,R5 ;Save word: MOV R0,
(R5)+ DECR R3 ;Bump word counter BNEQ @@loop
;Continue for all words JR R2 ;Return to caller: MOVR
R2,PC ENDP
1006
```

I/O

As was common for the era, the CP1600 used [memory-mapped I/O](#), as opposed to separate I/O pins as seen on the Intel systems. The use of a multiplexed bus and multi-state bus status made implementing I/O more difficult than would normally be the case on memory-mapped systems. This meant that implementations had to use latches or buffers to be able to interface with the CPU

as it changed the bus from indicating an address to data.^[4] This both negatively affected I/O performance and increased the complexity of the I/O devices.

To address this problem, GI supplied a series of 164x dedicated I/O chips that implemented the required bus logic. These included, for instance, the 1641 keyboard controller, the 1643 cassette tape controller, and the 1647 display control. Most famous among these is the 1640 "[Programmable Interface Controller](#)", or PIC, which was designed to work in concert with the CP1600 and act as a [channel controller](#) for the CPU. As with the other 1640 series chips, the PIC internally decoded the bus logic, but also added a very simple processor that could run its own programs to perform I/O and [direct memory access](#). For instance, one might send an instruction to a PIC on a [floppy disk](#) card to read data from a given sector on the disk. The PIC would then read the data into its own internal buffer, watch the bus for unused time when the bus status pins were all zero, and then send data to main memory.

General Instrument provided cross-assemblers and simulators/debuggers compatible with 16-bit or larger [minicomputer](#)s.^[5] GI also provided a standalone CP1600 based microcomputer system in the GIC1600.^[6]

Uses

The CP1610, used in game consoles such as the [Champion 2711](#)^[7] and most notably the [Intellivision](#), is a compatible member of the 1600 microprocessor family. It uses a 2 MHz two-phase clock producing a 1 microsecond processor cycle. The CP1610 in the [NTSC](#) Intellivisions uses a 1.7897725 MHz two-phase clock. Although users of the CP1600 in the traditional computer role were relatively rare, over 3 million Intellivisions were produced from 1980 until the [video game crash of 1983](#) led to the closing of the Intellivision production lines in 1984.^[8]

Production of the CP1600 ended in 1985 when General Instrument spun off its microelectronics division to create [Microchip Technology](#). By this point a number of 32-bit designs like the [MC68000](#) were available that limited interest in a 16-bit design like the CP1600, and their main existing customer, the Intellivision, was no longer in production. Many other products were also end-of-lifed at the same time, and their primary product was the PIC.

References

Bibliography

- Book: Osborne . [Osborne 16-Bit Microprocessor Handbook](#) . Osborne/McGraw-Hill . 1981 . 0-931988-43-8 .
- Book: [Series 1600 Microprocessor System](#) . General Instrument . 1975 . CITEREFSeries16001975.

External links

- "[CP1610](#)" at the [Intellivision Wiki](#)

Notes and References

1. Web site: [General Instrument's microprocessor aimed at minicomputer market](#) . January 2000 .
2. Book: Belzer . Jack . Holzman . Albert G. . Kent . Allen . [Encyclopedia of Computer Science and Technology: Volume 10 - Linear and Matrix Algebra to Microorganisms: Computer-Assisted Identification](#) . 1978 . . 9780824722609 . 402 .
3. Book: [CP-1600 Microprocessor Users Manual](#) . May 1975 . General Instruments . S16DOC-CP 1600 -04 . 5 July 2022.
4. Lowell Turner, "[General Instruments CP1600](#)", 10 July 2001
5. Book: General Instrument . November 1974 . [CP-1600 Cross Assembler Simulator Users Manual](#) .
6. Book: General Instrument . September 1975 . [GIC1600 Microcomputer Users Manual](#) .
7. Web site: [Champion 2711 Pre-83](#) . 2022-05-23 . pre83.com.
8. Web site: [Mattel Intellivision - 1980-1984](#) . ClassicGaming . . 2008-05-16 . dead . https://web.archive.org/web/20080623232114/http://classicgaming.gamespy.com/View.php?view=ConsoleMuseum.Detail&id=17&game=9 . 2008-06-23 .